Accelerating Test-Time Discovery in Complex Code Generation via Execution-Guided Self-Distillation

Mai Phong Đăng

VNUHCM University of Science
Supervisor: Dr. Ngô Minh Mẫn

July 2026

The problem

A model must **solve a hard problem it currently fails** — improving only from *its own attempts* and a verifiable *pass/fail* reward.

Two obstacles:

- Sparse signal: the reward says only *whether* an attempt passed, not *what* was wrong — hard to know which part to fix (credit assignment).
- Nothing to imitate: on a truly hard problem the model **rarely produces any correct attempt** — so there is no correct example to learn from.

⇒ **Flat-reward trap:** no correct attempt ⇒ no usable gradient ⇒ learning stalls on exactly the hardest problems — the ones we care about.

*Goal: accelerate this **discovery** on complex code generation.*

Background: SDPO and the gap

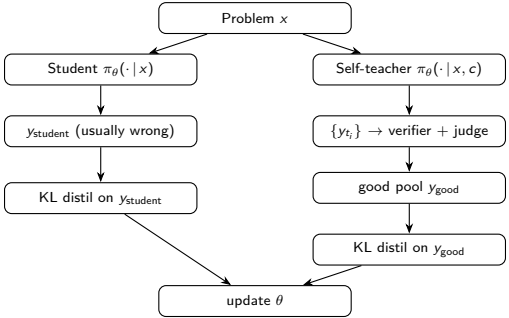
Setting — test-time training (TTT): fix one hard problem, let the model improve over a few inference-time steps, then reset; measure how fast it **discovers** a solution (discovery@k).

SDPO (Hübotter et al.) attacks the *sparse-signal* obstacle: turn the *rich feedback* verifiable environments already produce (failing tests, runtime errors) into a **dense** signal.

- The feedback-conditioned model is a **self-teacher**; its per-token distribution is distilled into the student.
- Denser credit assignment than GRPO: a K-dim target per token, not one scalar per sequence.

The gap: SDPO is student-first — it distills the student's *own failed rollout*. On hard problems little is correct to learn from $\Rightarrow$ it still hits the flat-reward trap. $\leftarrow$ **our target.**

Our idea: teacher-first



Teacher-first: the self-teacher generates under *privileged context* c (feedback + few-shot); a **verifier** + **judge** keep only *correct AND independent* solutions (y_{good}), and the student is distilled toward those. Sits **between** on-policy SDPO and off-policy SFT; the clean filter **never distils a copy** (all copies ⇒ empty pool ⇒ that step distils nothing).

Teacher-first: the loss

Per-token KL — pull the *student* toward the *self-teacher*'s (context-conditioned) next-token distribution, summed over the filtered good pool $\mathcal{G}$:

$$\mathcal{L}_{\mathrm{TF}}(\theta) = \sum_{y \in \mathcal{G}} \sum_{t=1}^{|y|} \mathrm{KL}\Big(\underbrace{\pi_\theta(\cdot \mid x, y_{<t})}_{\text{student}} \;\big\|\; \mathrm{stopgrad}\, \underbrace{\pi_\theta(\cdot \mid x, c, y_{<t})}_{\text{self-teacher}} \Big)$$

- **stopgrad** on the teacher: only the student updates — stops the teacher collapsing onto it and ignoring c.
- Gradient per logit: $\partial \mathcal{L} / \partial z_v = \pi_S(v) - \pi_T(v)$ — raise every token the teacher trusts more than the student.
- vs. student-first SDPO: *same form*, but sum over the correct y_{good} instead of the student's *failed* y_{student}.

Implemented as top-K reverse KL (K=20); teacher = same weights run under `no_grad` (self-distillation).

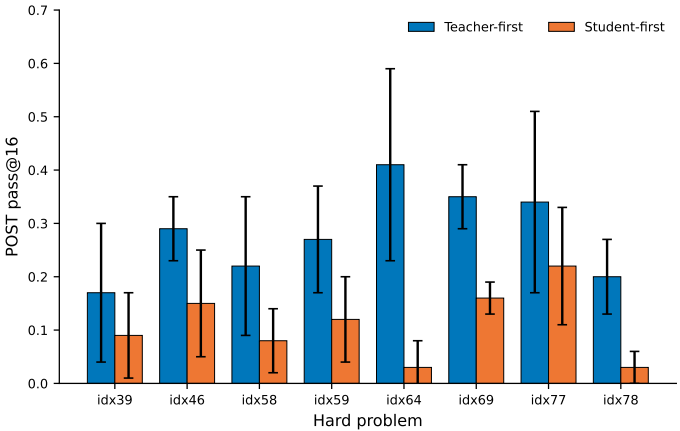
Research objectives

- **RO-method:** Establish whether teacher-first beats student-first at discovering hard code solutions — and whether it holds at *matched compute*.
- **RO1:** Determine whether the *reprompt-template* formulation of the feedback affects discovery.
- **RO2:** Measure whether distillation from rich context *suppresses epistemic verbalization* on code.
- **RO3:** Characterize the *structural & domain limitations* going from procedural code to value-only math.

Setup

- Model: **Qwen3-4B** (LoRA), thinking ON — the *same* model on both code (LiveCodeBench v6) and math (AIME 2026 pilot).
- Per problem: reset to base, **15 TTT steps**, evaluate student PRE vs POST.
- **8 frontier code problems** $\times$ **6 seeds** (medium scale) $\Rightarrow$ Wilcoxon signed-rank test.
- Metrics: pass@16, discovery@k, total generations + GPU-time.

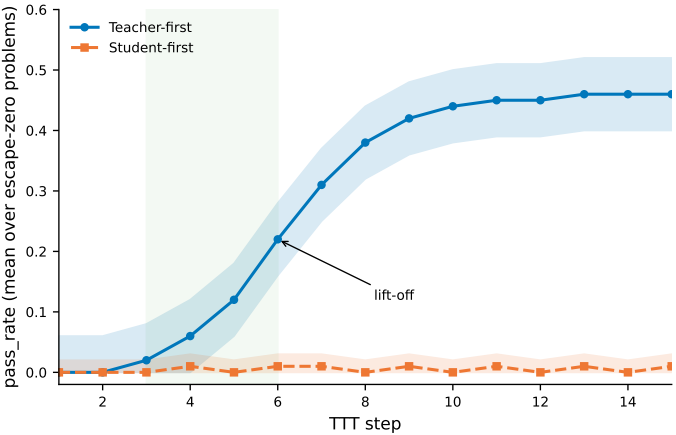
RO-method: teacher-first wins



Mean over 6 seeds; error bars = 1 SD. Wilcoxon signed-rank $p < 0.01$.

- TF $>$ SF on every problem (mean), largest margins on the hardest — **39 / 8 / 1** over 48 seed-pairs.
- Wilcoxon $p < 0.01$ (8 problems $\times$ 6 seeds) — not a crude sign test.
- Escape-zero cases (idx64: **0.41** vs **0.03**) anchor the comparison.

The headline: escape-zero



Student-first stays flat at 0; teacher-first escapes from step ∼4.

- Student-first stays flat at zero.
- Teacher-first **lifts off** from ∼step 4.
- Direct signature of the flat-reward trap: on-policy has nothing correct; off-policy injection does.

RO1: the reprompt template shapes discovery

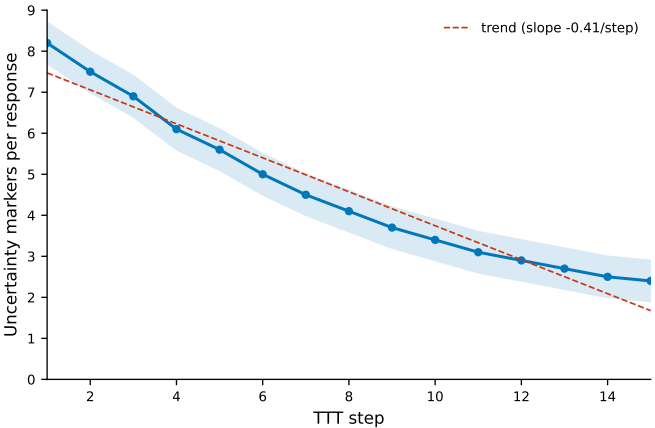
The template sets *what feedback the teacher sees*, so it directly changes the distillation target. We probe three points: **T1** (minimal), **T2** (anchor), **T5** (reasoning-inducing).

Template	idx12 (easy)	idx39 (hard)
T1 minimal	0.69	0.04
T2 anchor	0.94	0.07
T5 reasoning	0.95	0.12

POST pass@16, 6 seeds

- On hard problems: T5 $>$ T2 $>$ T1 — reasoning-inducing discovers best (significant).
- On easy problems the templates **saturate** (all near ceiling).
- Largest **marginal utility** when the reasoning template is paired with teacher-first on the hardest problems.

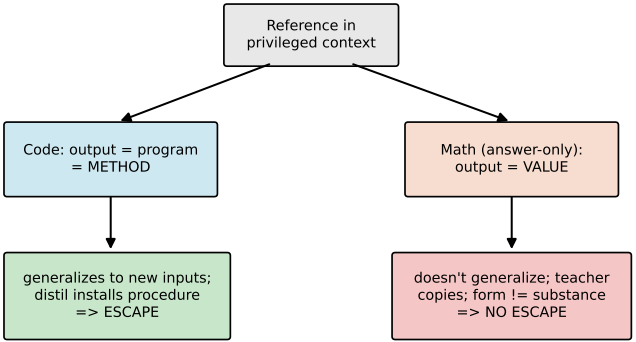
RO2: epistemic verbalization on code



Epistemic markers decline and accumulate downward across TTT steps (code, thinking ON).

- Uncertainty markers ("wait", "let me reconsider") **decline** across steps, accumulating.
- Consistent with Kim et al.: high $I(y; c \mid x) \Rightarrow$ suppression.
- *On code* it co-occurs with improved correctness — read as consolidation (a *hypothesis*; ref-type ablation is future work).

RO3: value vs. procedure — when it works



- Code output *is* a method (a program that generalizes) ⇒ distillation installs a procedure ⇒ **escape**.
- Answer-only math is a value ⇒ teacher can only copy ⇒ **no escape**.
- *Same* Qwen3-4B on both ⇒ not a weaker model; reference type is the *primary* difference (budget also differs) — n=2 pilot, read as directional.

Contributions

1. A **teacher-first** organization of execution-guided self-distillation (clean filter, never distils a copy).
2. Teacher-first **significantly accelerates discovery** and **escapes the flat-reward trap** — compute-fair (RO-method).
3. A **reprompt-template effect**, strongest on hard problems (RO1).
4. A measured **epistemic-suppression** result on code (RO2).
5. A **value-versus-procedure boundary** characterizing *when* the method helps.

Limitations & future directions

Honest bounds (§6.2):

- One **4B** Qwen model at a personal compute scale — a characterization, not a scaling law.
- Math pilot bottlenecked by **AIME**'s answer-only feedback (no worked solutions to derive from); n=2.
- A stronger base (**8B+**) is expected to *narrow* the TF-vs-SF margin on reachable tasks.
- Epistemic result tied to one model / lexical marker set.

Future directions: method-bearing math references (**MATH-500**); a same-domain *reference-type ablation* to test the boundary causally; scale to **8B+** and a second code benchmark.

Conclusion

- **Teacher-first execution-guided self-distillation** accelerates test-time discovery in complex code generation.
- It escapes the flat-reward trap, the advantage is **compute-fair**, and it shows a regime-dependent epistemic effect.
- The **value-versus-procedure boundary** explains *when* it works: an in-reach method, not just an answer.

Thank you

for your attention.